

Profit Leak Hunter — AI Business Auditor

Field	Value
Version	1.0
Status	Draft for Hackathon / Unicorn Track Submission
Owner	[Your Name]
Last Updated	July 6, 2026

1. Executive Summary

Profit Leak Hunter is an AI-powered business auditor that ingests raw operational data — sales, expenses, inventory, refunds, and supplier records — and automatically surfaces **hidden sources of profit loss** that owners can't see in spreadsheets or dashboards.

Instead of showing charts and asking the user to interpret them, Profit Leak Hunter reads the data like a forensic accountant and hands back plain-English findings:

“Your refund rate increased 18% in Product A over the last quarter.”
“Supplier B has quietly raised costs, cutting your margin by 6%.”
“Your discount campaigns are reducing profit faster than they're growing revenue.”

The product's core insight: **most small and mid-size businesses don't lack data — they lack someone to read it critically.** Profit Leak Hunter is that reader.

2. Problem Statement

2.1 The Problem

Small and mid-size businesses generate transactional data constantly (POS exports, accounting software, supplier invoices, e-commerce refund logs) but almost never analyze it beyond top-line revenue and expenses. As a result:

- Margin erosion from suppliers goes unnoticed for months.
- Refund/return spikes on specific products are invisible until they've already cost thousands.
- Discounting strategies are evaluated on “did sales go up” rather than “did profit go up.”
- Inventory that quietly ties up cash or spoils/expires is discovered too late.

2.2 Why This Is Hard to Solve Today

- **BI tools (Tableau, Power BI, Looker)** require the user to already know what question to ask. They visualize; they don't diagnose.
- **Accounting software (QuickBooks, Xero)** reports what happened, not why it's a problem or what's causing it.
- **Hiring an analyst/fractional CFO** is expensive and not accessible to small businesses.

2.3 The Opportunity

An AI system that behaves like an auditor — proactively finding anomalies, ranking them by financial impact, and explaining root causes in business language — closes this gap without requiring the user to know what to look for.

3. Target Users

Segment	Description	Primary Need
Small business owners	Retail, restaurants, e-commerce (1-50 employees)	“Where am I losing money without knowing it?”
E-commerce sellers	Shopify/Amazon sellers with refund & supplier complexity	Margin protection, refund root-cause
Fractional CFOs / bookkeepers	Manage multiple client businesses	Faster diagnostic tool to service more clients
Franchise / multi-location operators	Need to compare location-level performance	Cross-location leak detection

Primary persona for MVP/demo: An independent e-commerce or retail business owner who exports CSVs from Shopify, QuickBooks, or their POS system and has no in-house data analyst.

4. Goals & Non-Goals

4.1 Goals

- Detect and rank profit leaks across 4 core categories: **refunds, discounts, supplier/COGS, inventory**.
- Translate statistical findings into causal, plain-English narratives a non-technical owner understands.
- Quantify each finding in **dollar impact**, not just percentage change, so leaks can be prioritized.
- Deliver a working demo in under [X] hours using realistic synthetic or real anonymized CSV data.

4.2 Non-Goals (MVP)

- Real-time data pipeline integrations (Shopify/QuickBooks live APIs) — CSV upload only for MVP.
- Full forecasting or predictive modeling (this is a diagnostic tool, not a forecasting tool).
- Multi-user team collaboration features.
- Mobile app (web-first).

5. Success Metrics

Metric	Target (Demo/MVP)
Time from CSV upload to first insight	< 30 seconds
Number of distinct leak categories detected per dataset	≥ 3
Insight traceable to real underlying data pattern (precision)	$\geq 90\%$ on demo dataset
Judge/user comprehension (“I understood the problem in one sentence”)	Qualitative, gathered live
Estimated \$ impact surfaced per business	Framed prominently in UI

6. Core Product Features

6.1 Data Ingestion Layer

- CSV upload (drag-and-drop) for up to 4 data types:
 - **Sales/orders** (date, product, quantity, price, revenue)
 - **Refunds/returns** (date, product, reason code if available, refund amount)
 - **Supplier/COGS** (supplier name, product, unit cost, date, purchase volume)
 - **Inventory** (SKU, stock level, cost basis, date)
- Auto-detect column mapping using an LLM-assisted schema inference step (handles messy/inconsistent headers).
- Validation layer: flags missing fields, duplicate rows, currency mismatches before analysis runs.

6.2 Leak Detection Engine (Core IP)

A multi-pass statistical + reasoning pipeline that runs independently across each category:

1. Refund Leak Detector
 - Computes refund rate by product/category over time windows (week-over-week, month-over-month).
 - Flags statistically significant increases (e.g., z-score or rolling baseline comparison).
 - Cross-references refund spikes against price changes, supplier changes, or seasonality to suggest causes.
2. Supplier Margin Detector
 - Tracks unit cost per supplier per product over time.
 - Detects creeping cost increases that haven't been reflected in sale price (margin compression).
 - Ranks suppliers by cumulative margin impact.
3. Discount/Promotion Efficiency Detector
 - Compares revenue lift vs. margin loss for discounted periods/products.
 - Flags promotions where the discount cost exceeded the incremental profit generated (net negative ROI discounting).
4. Inventory Leak Detector
 - Flags slow-moving or dead stock tying up capital.
 - Flags shrinkage/loss patterns (stock decreasing without matching sales).

Each detector outputs structured findings:

```
{
  "category": "refund",
  "entity": "Product A",
  "metric_change": "+18%",
  "dollar_impact": -4200,
  "confidence": 0.87,
  "time_window": "Q2 2026 vs Q1 2026",
  "likely_cause": "correlates with a price increase applied 3 weeks prior"
}
```

6.3 AI Reasoning & Report Generation Layer

- Structured findings from the detection engine are passed to an LLM (e.g., Gemma or Qwen) as **grounding data**, not as the source of truth — this keeps the numbers trustworthy and the LLM's job strictly narrative/explanatory.
- LLM generates:
 - A ranked “Top 3 Profit Leaks” executive summary.
 - Plain-English causal narrative per finding.
 - Suggested next actions (“Renegotiate with Supplier B” / “Investigate Product A refund reasons”).
- Output rendered as a shareable report (PDF/HTML) and an in-app dashboard.

6.4 Dashboard / UI

- Landing view: “Your Top Profit Leaks This Period” — ranked by dollar impact, not just percentage.
- Drill-down view per leak: chart of the underlying trend + narrative explanation + suggested action.
- Upload history / comparison across periods (for repeat use).

6.5 Optional/Bonus Features

- **Multi-agent architecture with X402:** Specialized detector agents (refund, supplier, discount, inventory) each independently priced and callable — demonstrating a functioning agent economy where an orchestrator agent “pays” specialist agents per analysis run.
- **Gemma/Qwen local reasoning:** Use an open-weight model for report generation to reduce cost and demonstrate independence from closed APIs — good judge talking point re: cost efficiency and data privacy.
- Benchmark comparison (“businesses like yours typically have a refund rate of X%”).

7. User Flow (MVP Demo Path)

5. User lands on homepage → “Upload your business data” (sample datasets provided for demo).
6. User uploads 2-4 CSVs (sales, refunds, supplier, inventory).
7. System auto-maps columns, shows a quick confirmation screen (“We detected: Date, Product, Refund Amount — correct?”).
8. Processing screen (5-30 sec) — detection engine runs.
9. Results dashboard: “You have 3 profit leaks costing you approximately \$X,XXX this quarter.”
10. User clicks into each leak → sees chart, narrative, suggested action.
11. User can export a PDF report or share a link.

8. Technical Architecture (High-Level)

```
CSV Upload → Schema Inference (LLM-assisted) → Data Normalization
  → Detection Engine (rule-based + statistical, per category)
  → Structured Findings (JSON)
  → LLM Reasoning Layer (Gemma/Qwen) → Narrative Report
  → Dashboard Render (Web UI) + PDF Export
```

Optional agent-economy layer (X402):

```
Orchestrator Agent
├─ pays → Refund Analysis Agent
├─ pays → Supplier Margin Agent
├─ pays → Discount Efficiency Agent
├─ pays → Inventory Agent
→ aggregates paid results → passes to Reasoning Layer
```

Confirmed stack:

- **Frontend:** React / Next.js + Tailwind (dashboard, charts via Recharts)
- **Backend:** FastAPI (Python) — handles CSV ingestion, validation, and serves detection results to the frontend via REST/streaming endpoints
- **Core AI/Agent Logic:** LangChain, LangGraph, and CrewAI, with LLMs (Gemma/Qwen and/or hosted models) selected per task — orchestration framework chosen based on which detection/reasoning step it fits best (see 8.1 below)

- **Detection logic:** pandas/numpy inside FastAPI services, called as tools by the agent layer rather than left as standalone scripts
- **Storage:** Local/session-based for demo (no persistent DB required for MVP)
- **Optional:** X402 for agent micropayments if pursuing the multi-agent bonus

8.1 Agent Framework Responsibilities

Since LangChain, LangGraph, and CrewAI each solve a different problem, it's worth being explicit about which one does what — this also gives a clean technical story for judges instead of “we used three frameworks because we could.”

Framework	Role in Profit Leak Hunter
LangChain	Tool/utility layer — wraps the pandas-based detectors (refund, supplier, discount, inventory) as callable tools, handles LLM prompt templates for schema-inference and narrative-generation, and manages structured output parsing (JSON findings → validated schema).
LangGraph	Orchestration/state machine for the end-to-end pipeline: Upload → Schema Inference → Parallel Detection → Aggregation → Reasoning → Report. Gives explicit control over branching (e.g., skip inventory node if not uploaded) and makes the pipeline debuggable and demoable as a graph.
CrewAI	Only invoked for the multi-agent bonus: models each detector as a distinct “crew member” agent with its own role/goal, coordinated by an orchestrator/manager agent. This is where the X402 payment-between-agents story plugs in naturally.

Recommended MVP simplification: Use LangGraph as the primary backbone (the natural fit for a linear-but-branching pipeline like this) with LangChain tools underneath. Only bring in CrewAI if there's time for the true multi-agent/X402 stretch goal — running all three frameworks for the MVP core adds integration risk for judges who will mainly care that the pipeline works and the findings are accurate.

9. Data Requirements

Minimum viable columns per file type:

File	Required Columns
Sales	date, product/SKU, quantity, unit_price, revenue
Refunds	date, product/SKU, refund_amount, (optional: reason)
Supplier/COGS	date, supplier_name, product/SKU, unit_cost
Inventory	date/snapshot, SKU, stock_level, unit_cost

For the demo, build a realistic synthetic dataset with intentionally seeded leaks (e.g., a supplier cost creep in month 3, a refund spike on one SKU, one clearly unprofitable discount campaign) so the tool's findings are verifiable live in front of judges.

10. Risks & Mitigations

Risk	Mitigation
Messy/inconsistent real-world CSVs break parsing	Build a robust, LLM-assisted schema-mapping step; provide clean sample data as fallback for live demo

Risk	Mitigation
LLM hallucinates causal explanations not supported by data	Keep LLM strictly bounded to reasoning over structured, pre-computed findings — never let it invent numbers
Judges question statistical validity of “leak” detection	Use transparent, explainable methods (rolling baselines, z-scores) and show the underlying chart, not just the claim
Scope creep before hackathon deadline	Lock MVP to 4 detectors + CSV upload + dashboard; treat X402/multi-agent as stretch goal only

11. Why This Fits the Unicorn Track

- **Real, relatable pain point:** every business owner has felt “money is leaking somewhere, I just don't know where.”
- **Demoable in minutes:** upload → insight, with dollar figures judges immediately understand.
- **Founder-market fit:** direct personal experience running businesses gives credible, specific examples during pitch.
- **Extensible technical depth:** the base product is understandable to non-technical judges, while the optional Gemma/Qwen + X402 multi-agent layer gives technical judges a differentiated architecture story.
- **Clear monetization path:** SaaS subscription for ongoing monitoring, usage-based pricing per report, or a per-agent-call model if the X402 architecture is built out.

12. MVP Scope Summary (What to Actually Build for the Hackathon)

Must-have:

- CSV upload (sales + refunds minimum; supplier/inventory as time allows)
- Refund leak detector + Discount efficiency detector (highest “wow” factor, easiest to seed convincingly in synthetic data)
- LLM-generated narrative report
- Simple dashboard UI with ranked, dollar-quantified findings

Should-have (if time allows):

- Supplier margin detector
- Inventory leak detector
- PDF export

Stretch (only if core is done early):

- X402 multi-agent architecture
- Local Gemma/Qwen inference for reasoning layer
- Benchmark comparisons against industry averages

13. Open Questions

- Which LLM will actually generate reports at demo time — hosted API or local Gemma/Qwen for the privacy/cost story?
- Will the demo use fully synthetic data, anonymized real data from your own businesses, or both?
- Is the X402 agent-payment layer a genuine functional requirement for this hackathon's track, or purely a narrative/bonus element for the pitch?